Institut Supérieur d'Informatique
Année Universitaire 2025–2026

RAPPORT DE PROJET

Conception et Implantation d'un Compilateur

NutriLang

Un langage dédié à la Nutrition et à la Santé

Matière : Techniques de Compilation
Encadrant : Mme Hayet Mogaadi

Réalisé par :
Nour Chaker - Wiem Hamzaoui - Mazen Khoualdi

1. Introduction et Contexte du Projet

La compilation est l'une des disciplines fondatrices de l'informatique. Elle désigne l'ensemble des processus qui transforment un programme écrit dans un langage de haut niveau en une forme exécutable par une machine.
Au-delà de sa dimension technique, la compilation constitue un pont entre la pensée humaine exprimée sous forme de code source et le calcul mécanique.
Dans le cadre du module Techniques de Compilation dispensé à l'Institut Supérieur d'Informatique, il nous a été demandé de concevoir et d'implanter un compilateur complet pour un langage de notre choix.
Ce projet mobilise plusieurs outils issus de la chaîne de compilation classique : Flex pour l'analyse lexicale, Bison pour l'analyse syntaxique, C pour l'exécution sémantique, et une interface web (Flask + HTML/CSS/JS) pour rendre l'environnement accessible à tout utilisateur.

2. Fonctionnalités principales

— **Déclaration de variables :** Attribution de valeurs numériques et de chaînes à des identifiants.
— **Gestion des repas :** Déclaration d'un repas avec son nom et son contenu calorique via le mot-clé Repas.
— **Calcul de l'IMC :** Primitives Calculer_IMC et Verifier_healthy pour évaluer l'état nutritionnel.
— **Objectifs diététiques :** Trois valeurs possibles : perte_poids, prise_muscle, maintien.
— **Suggestions intelligentes :** La commande Suggestion_healthy génère des recommandations selon l'objectif.
— **Affichage nutritionnel :** La commande afficher_nutrition produit un rapport nutritionnel complet.
— **Structures de contrôle :** Blocs conditionnels (Si/Alors/Sinon/Finsi) et boucles (Tantque/Fait).

— **Opérateurs logiques :** Opérateurs ET, OU, NON pour les expressions booléennes.
— **Conseils nutritionnels :** Affichage de conseils personnalisés via la commande Conseil.

3. Architecture Générale du Compilateur

Le compilateur NutriLang adopte une architecture en pipeline classique, composée de quatre phases principales.
Chaque phase transforme la représentation du programme d'une forme vers une autre, en validant des propriétés de plus en plus abstraites.

3.1 Vue d'ensemble du pipeline

Phase 1 — Analyse Lexicale
Le code source est découpé en tokens (unités lexicales).
Outil : Flex. Fichier : langage1.l
Phase 2 — Analyse Syntaxique
Les tokens sont organisés selon une grammaire formelle.
Outil : Bison. Fichier : langage1.y
Phase 3 — Analyse Sémantique & Exécution
Les actions C associées aux règles de grammaire évaluent les expressions, gèrent les variables et produisent les sorties.
Phase 4 — Interface Utilisateur
Un serveur Flask sert une page web permettant d'éditer et d'exécuter du code NutriLang.

3.2 Structure du projet

L'organisation des fichiers du projet reflète la séparation des préoccupations entre la partie compilateur et la partie interface web :

```
NUTRILANG/
■■■ langage1.l          ← Analyseur lexical (Flex)
■■■ langage1.y          ← Analyseur syntaxique (Bison)
■■■ compile.sh          ← Script de compilation automatique
■■■ nutrilang           ← Exécutable généré
■■■ web-ide/
    ■■■ server.py       ← Serveur Flask
    ■■■ templates/
    ■   ■■■ index.html  ← Interface utilisateur
    ■■■ static/
        ■■■ images/     ← Images des suggestions de repas
```

4. Analyse Lexicale (Flex)

4.1 Rôle de l'analyse lexicale

L'analyse lexicale est la première phase du compilateur. Elle lit le code source caractère par caractère et le découpe en une suite de tokens , les plus petites unités signifiantes du langage.
Chaque token est associé à un type (mot-clé, identifiant, nombre, opérateur...) et éventuellement à une valeur.
Dans NutriLang, l'analyseur lexical est généré par Flex à partir du fichier langage1.l.
Ce fichier définit des expressions régulières associées à des actions C.

4.2 Catégories de tokens

Catégorie	Exemples de tokens	Description
Mots-clés de contrôle	Si, Alors, Sinon, Finsi, Tantque, Fait	Structures conditionnelles et boucles
Primitives métier	Repas, Calculer_IMC, Verifier_healthy, Conseil	Fonctions spécifiques au domaine
Objectifs	perte_poids, prise_muscle, maintien	Valeurs d'objectif nutritionnel
Opérateurs logiques	ET, OU, NON	Logique booléenne
Opérateurs comparaison	==, >=, <=, >, <	Comparaisons numériques
Opérateurs arithm.	+, -, *, /	Calculs numériques
Affectation	:=	Assignation de valeur
Nombres	42, 3.14, 1.8	Entiers et décimaux
Identifiants	poids, imc, mon_repas	Noms de variables
Chaînes	"Salade César"	Texte entre guillemets
Commentaires	// texte...	Ignorés par le compilateur

Voici les règles lexicales les plus représentatives du fichier Flex :

```
"Si"                    { return SI; }
"Alors"                 { return ALORS; }
"Sinon"                 { return SINON; }
"Repas"                 { return REPAS; }
"Calculer_IMC"          { return CALCULER_IMC; }
"perte_poids"           { return PERTE_POIDS; }
[0-9]+(\.[0-9]+)?       { yylval.reel = atof(yytext); return NB; }
[a-zA-Z][a-zA-Z0-9_]*   { yylval.nom = strdup(yytext); return ID; }
"([^"\\]|\\.)*"   { yylval.nom = strdup(yytext+1);
                          yylval.nom[strlen(yylval.nom)-1] = '\0';
                          return STRING; }
"//".*                  { ; }   /* commentaires ignores */
[ \t\r\f]+              { ; }   /* espaces ignores */
```

4.4 Gestion des erreurs lexicales

Tout caractère non reconnu par aucune règle déclenche l'affichage d'un message d'erreur via la règle fourre-tout . (point), qui correspond à n'importe quel caractère unique. Cette règle est placée en dernier dans le fichier .l afin d'avoir la plus basse priorité :

```
.                       { printf("Erreur lexicale : [%s]\n", yytext); }
```

5. Analyse Syntaxique (Bison)

5.1 Rôle de l'analyse syntaxique

L'analyse syntaxique prend en entrée le flux de tokens produit par l'analyseur lexical et vérifie que leur enchaînement respecte la grammaire du langage.
Elle construit implicitement un arbre syntaxique abstrait (AST) dont les nœuds correspondent aux constructions du langage.
NutriLang utilise Bison, un générateur d'analyseurs syntaxiques ascendants (LALR(1)).
La grammaire est définie dans le fichier langage1.y, qui associe à chaque règle de production une action sémantique écrite en C

5.2 Règles de grammaire principales

La grammaire de NutriLang est organisée autour de plusieurs catégories syntaxiques :
programme : Séquence d'instructions séparées par des retours à la ligne.
instruction : Affectation, condition, boucle, déclaration de repas, calcul, affichage ou objectif.
expression : Combinaison d'opérations arithmétiques, de comparaisons et d'opérateurs logiques.
bloc_si : Bloc conditionnel complet : Si ... Alors ... [Sinon ...] Finsi.
boucle : Boucle Tantque ... Fait. repas : Déclaration d'un repas : Repas "nom" calories.

5.3 Union de valeurs sémantiques

La directive %union de Bison déclare les types de valeurs que peuvent porter les tokens et les non-terminaux. Dans NutriLang, deux types sont utilisés :

```
%union {
    double reel;
    char *nom;
}
```

5.4 Gestion des conflits et priorités

Les grammaires arithmétiques introduisent naturellement des ambiguïtés. Bison les résout grâce aux directives de précédence et d'associativité déclarées en en-tête du fichier .y. Pour NutriLang, la hiérarchie est la suivante (de la plus faible à la plus forte priorité) :

Niveau 1 (plus faible)	OU	Disjonction logique
Niveau 2	ET	Conjonction logique
Niveau 3	NON	Négation (unaire)
Niveau 4	< > <= >= ==	Comparaisons
Niveau 5	+ -	Addition, soustraction
Niveau 6 (plus forte)	* /	Multiplication, division

6. Sémantique et Exécution (C)

6.1 Table des symboles

La table des symboles est la structure de données centrale du compilateur. Elle associe chaque identifiant (nom de variable) à sa valeur courante. Dans NutriLang, elle est implémentée en C sous la forme d'un tableau de structures .

6.2 Calcul de l'IMC

La primitive Calculer_IMC attend que les variables poids et taille soient définies dans la table des symboles. Elle calcule l'IMC selon la formule standard IMC = poids / (taille × taille) et stocke le résultat dans la variable imc .

6.3 Vérification de la santé nutritionnelle

La commande Verifier_healthy classe l'IMC calculé selon les seuils établis par l'Organisation Mondiale de la Santé :

Plage IMC	Classification OMS	Message affiché
< 18.5	Insuffisance pondérale	Maigreur détectée
18.5 – 24.9	Corpulence normale	IMC sain — Bravo !

25.0 – 29.9	Surpoids	Surpoids détecté
≥ 30.0	Obésité	Obésité détectée

6.4 Gestion des objectifs nutritionnels

La commande Objectif enregistre un objectif diététique de l'utilisateur. La commande Suggestion_healthy exploite ensuite cet objectif pour afficher des recommandations de repas adaptées.

Trois objectifs sont disponibles :

perte_poids : Suggestions riches en protéines, faibles en graisses saturées (ex : salade de poulet, poisson brocoli).

prise_muscle : Suggestions caloriques et protéinées (ex : shake banane, poulet-riz).

maintien : Suggestions équilibrées (ex : bowl saumon, lentilles corail)

7. Interface Web — NutriLang Studio

7.1 Présentation générale

NutriLang Studio est l'environnement de développement intégré (IDE) web fourni avec le compilateur. Il permet à un utilisateur d'écrire un programme NutriLang dans un éditeur de code en ligne, de l'exécuter et de visualiser les résultats sans aucune installation locale.

7.2 Architecture du serveur Flask

Le backend est développé avec le micro-framework Flask (Python). Il expose deux routes principales :

GET / : Sert la page HTML principale de l'IDE.

POST /execute : Reçoit le code NutriLang en JSON, le passe en entrée standard à l'exécutable nutrilang via un sous-processus, et retourne la sortie standard au client.

7.3 Fonctionnalités de l'interface

— Éditeur de code avec coloration syntaxique

— Sélection d'exemples NutriLang prédéfinis

— Bouton d'exécution avec retour en temps réel

— Affichage de la sortie du compilateur

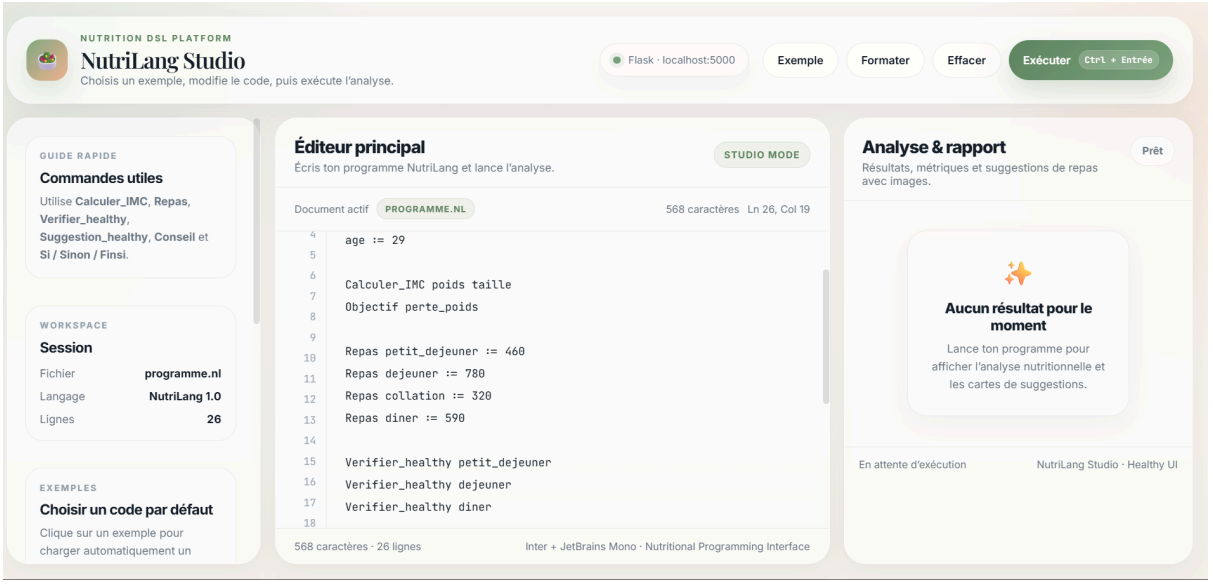
— Galerie de suggestions de repas avec images
— Design responsive, accessible sur mobile

7.5 Démarrage du projet

```
# 1. Créer l'environnement Python
python3 -m venv venv
source venv/bin/activate
pip install flask flask-cors

# 2. Compiler NutriLang
chmod +x compile.sh
./compile.sh

# 3. Lancer l'interface web
cd web-ide
python3 server.py
# Accès : http://localhost:5000
```



Analyse & rapport

Exécution réussie

Résultats, métriques et suggestions de repas avec images.

INDICE IMC

27.76

Surpoids

CALORIES REPAS

2150 kcal

4 repas détecté(s)

Suggestions healthy illustrées

Objectif : perte poids

Exécuté en 24 ms

NutriLang Studio · Healthy UI